

Stima della cardinalità distinta nei data stream distribuiti

Algoritmi di sketching, framework sperimentale e merge semanticamente corretto

Seminario magistrale

Daniele Ferroli

Università degli Studi di Udine

A.A. 2024–2025

Roadmap del seminario

- ① Problema, modello minimo e vincoli del count-distinct
- ② Stato dell'arte: da Probabilistic Counting a HyperLogLog++
- ③ Implementazione: architettura, hash, dataset e validazione
- ④ Disegno sperimentale in modalità streaming e merge
- ⑤ Risultati principali su accuratezza, robustezza e compatibilità semantica
- ⑥ Conclusioni, limiti e sviluppi futuri

Problema e contributo del lavoro

- Obiettivo: stimare quanti identificatori distinti compaiono in un flusso continuo senza conservare tutto l'input.
- Applicazioni tipiche: monitoraggio di rete, telemetria, osservabilità, analisi di log, ottimizzazione di interrogazioni.
- Nei sistemi distribuiti non basta stimare bene in locale: serve anche combinare sketch prodotti su nodi o partizioni diverse.
- La tesi non propone un nuovo stimatore teorico, ma uno studio sistematico del comportamento reale degli sketch e del loro merge.

Contributo principale

Implementazione uniforme di più algoritmi count-distinct, interfaccia comune per le hash, framework riproducibile e distinzione esplicita tra casi `valid`, `recoverable` e `invalid`.

Modello minimo: stream, F_0 e sketch

- Il flusso è una sequenza ordinata $S = \langle x_1, x_2, \dots, x_s \rangle$.
- Il lavoro assume il modello *insertion-only*: ogni aggiornamento aggiunge occorrenze, non le rimuove.
- Gli algoritmi devono lavorare in uno o pochi passaggi, con memoria sublineare e aggiornamenti rapidi.
- Nel framework esiste anche un riferimento esatto interno, NaiveCounting, usato per test e controlli.

$$F_0 = |\{a \in \mathcal{U} \mid f(a) > 0\}|$$

Interpretazione

F_0 è la cardinalità del supporto del vettore delle frequenze, cioè il numero di elementi distinti osservati.

Vincolo di fondo

Il conteggio esatto non scala bene quando il flusso cresce; da qui nasce l'uso di sketch probabilistici.

Evoluzione storica dello stato dell'arte

Metodo	Idea dominante	Ruolo nella tesi
Probabilistic Counting	Bitmap singola che osserva eventi rari sull'hash	Primo punto della linea storica e benchmark implementato
PCSA	Mediazione stocastica su più bitmap virtuali	Passo intermedio ricostruito nello stato dell'arte
LogLog	Registri e media geometrica dei massimi di rango	Benchmark implementato
HyperLogLog	Media armonica e correzioni di regime	Benchmark implementato e riferimento vicino a HLL++
HyperLogLog++	Bias correction + stato sparso/denso	Riferimento pratico principale per gli esperimenti di merge

Lo stato dell'arte richiama anche Bloom filter e Count-Min Sketch per mostrare che lo sketching si estende oltre il solo conteggio dei distinti.

Algoritmi implementati nel framework

ID	Metodo	Ruolo
naive	NaiveCounting	riferimento esatto interno
pc	Probabilistic Counting	benchmark pubblico
ll	LogLog	benchmark pubblico
hll	HyperLogLog	benchmark pubblico
hllpp	HyperLogLog++	benchmark pubblico

- Interfaccia comune con process, count, merge e reset.
- La funzione di hash è iniettata nel costruttore e fa parte del contesto dello sketch.
- HLL++ è l'implementazione più articolata: stato sparso/denso, tabelle di bias, riduzione esplicita alla precisione minima.

Sketch, hash e mergeabilità

- Uno *sketch* mantiene una sintesi compatta del flusso e permette di stimare F_0 senza conservare tutti gli elementi.
- Negli sketch count-distinct l'hash non serve solo a randomizzare: determina come il flusso viene codificato nello stato.
- Uno sketch è *mergeabile* se combinare due stati equivale a elaborare serialmente l'unione dei due flussi, a parità di semantica o con degradazione esplicita.

$$\hat{\theta}(\mathcal{K}(S_1) \oplus \mathcal{K}(S_2)) \approx \hat{\theta}(\mathcal{K}(S_1 \cdot S_2))$$

$$(\mathcal{K}(S_1) \oplus \mathcal{K}(S_2))[j] = \max\{\mathcal{K}(S_1)[j], \mathcal{K}(S_2)[j]\}$$

Punto critico

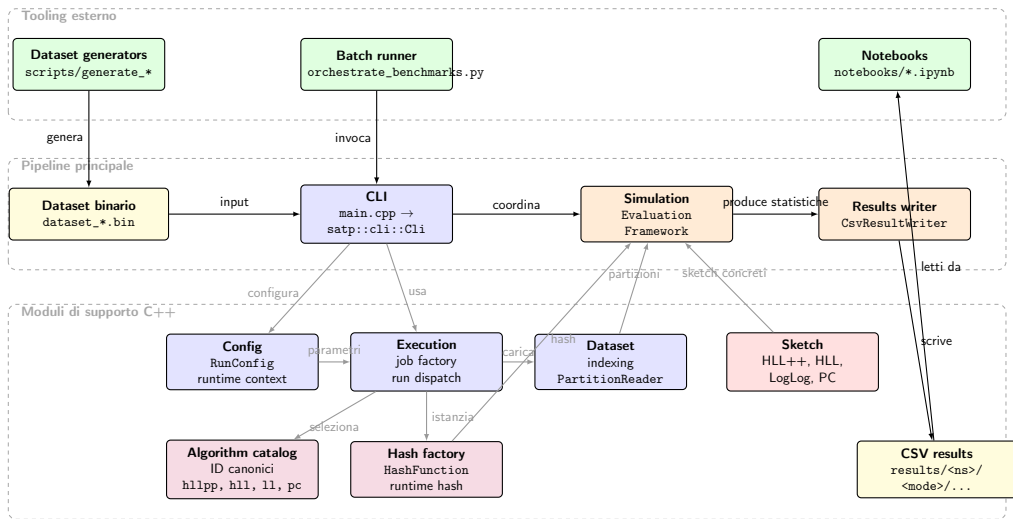
Per gli sketch HLL-like il merge a massimi è corretto solo se i due lati condividono parametri compatibili e la stessa semantica dell'hash.

Compatibilità semantica del merge

valid	recoverable	invalid
Il merge è corretto direttamente. Caso tipico: HLL++ con stessa precisione, stessa funzione di hash e stesso seed.	Il merge non è corretto subito, ma lo diventa dopo una normalizzazione esplicita. Caso tipico: HLL++ con precisioni diverse ma stessa semantica dell'hash.	In questo quadro non esiste un merge semanticamente corretto. Caso tipico: funzioni di hash o seed diversi.

- Strategia operativa distinta dalla compatibilità: `direct`, `reject`, `reduce_then_merge`.
- `unsafe_naive_merge` viene mantenuto solo come controllo negativo sperimentale.

Architettura generale del sistema



Tre strati: nucleo algoritmico, moduli di supporto (dataset e hash), coordinamento sperimentale (CLI, evaluation framework, writer CSV).

Hash, riproducibilità e scelte progettuali

Livello hash

- Interfaccia comune: `hash64`, `hash32`, `name`.
- Funzioni supportate: `splitmix64`, `xxhash64`, `murmurhash3`, `siphash24`.
- L'hash è parte della semantica dello sketch non un dettaglio di implementazione.

Riproducibilità

- Il seed del dataset è salvato nell'header binario ed è letto a runtime.
- Nel merge eterogeneo i seed di hash sinistro e destro possono essere configurati separatamente.
- La sessione distingue tra parametri utente e metadati derivati automaticamente dal dataset.

Dataset prefix_constant_rho e validazione

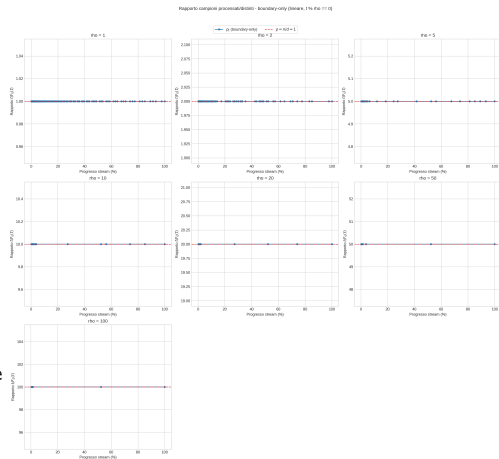
- Dataset sintetico controllato con $n = 10^7$, $p = 50$, seed = 21041998.
- Valori di $d \in \{10^5, 2 \cdot 10^5, 5 \cdot 10^5, 10^6, 2 \cdot 10^6, 5 \cdot 10^6, 10^7\}$.
- Rapporto globale

$$\rho = \frac{n}{d} \in \{100, 50, 20, 10, 5, 2, 1\}.$$

- Il generatore impone una nuova prima occorrenza per ogni blocco di lunghezza ρ , quindi

$$F_0(j \cdot \rho) = j.$$

- Il dataset salva un bitset di verità, così il framework ricostruisce $F_0(t)$ esatta sui prefissi.



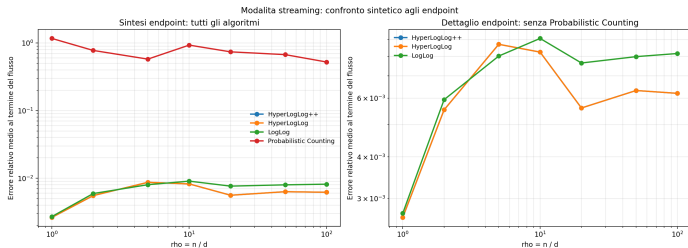
La validazione empirica conferma il comportamento atteso di $\rho_t = t/F_0(t)$ ai bordi di blocco.

Test dedicati coprono algoritmi, merge, framework, schema CSV e generatore del dataset.

Disegno sperimentale

- Ambiente di test: AMD Ryzen 7 9700X, 32 GB DDR5, Clang 22.1.1, CMake 4.3.0.
- Tre modalità di valutazione:
 - **streaming**: andamento della stima lungo i prefissi;
 - **merge omogeneo**: confronto tra merge e riferimento seriale;
 - **merge eterogeneo**: casi `valid`, `recoverable`, `invalid`.
- Due viste complementari in streaming:
 - caso A: stima in funzione della cardinalità reale del prefisso;
 - caso B: stima in funzione di $\rho_t = t/F_0(t)$ a cardinalità target fissata.
- Metriche principali: stima media, dispersione della stima, errore relativo medio ai punti finali.
- Gli esperimenti di merge sono concentrati su HyperLogLog++.

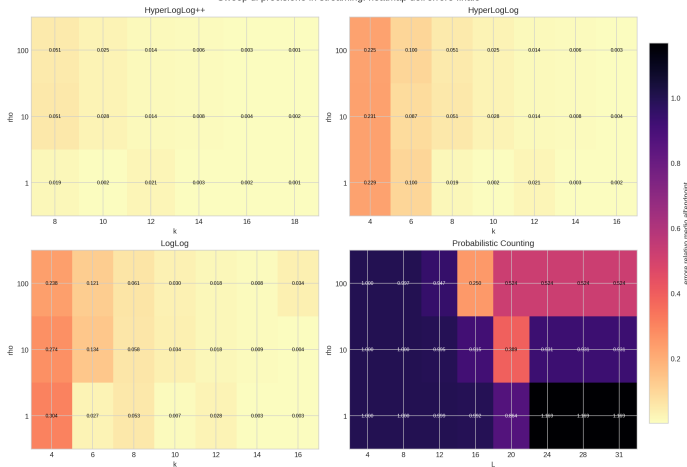
Risultato 1: comportamento degli algoritmi in streaming



- Nella vista su $F_0(t)$, HLL e HLL++ risultano quasi sovrapposti, con errore medio finale di circa 0.00618.
- Nella vista su ρ_t , HLL++ è il migliore con circa $3.95 \cdot 10^{-4}$, seguito da HLL con $2.08 \cdot 10^{-3}$.
- LogLog è più sensibile e nella seconda vista raggiunge circa 1.455.
- Probabilistic Counting resta molto instabile nel dominio osservato, con circa 0.770 nella prima vista e 0.346 nella seconda.

Risultato 2: l'aumento di precisione favorisce soprattutto HLL++

Sweep di precisione in streaming: heatmap dell'errore finale



Algoritmo	Migliore	Osservazione
HLL++	$k = 18$	migliore per tutti i valori di ρ osservati in media $k = 16$, ma con $\rho = 1$ basta $k = 10$
HLL	$k = 16$	andamento più irregolare; con $\rho = 10$ emerge $k = 16$
LogLog	$k = 14$	migliora poco; con $\rho = 100$ emerge $L = 16$

- HLL++ è il metodo con andamento più regolare e prevedibile.
- HLL migliora, ma meno uniformemente.
- LogLog e Probabilistic Counting mostrano dipendenza più marcata dal parametro.

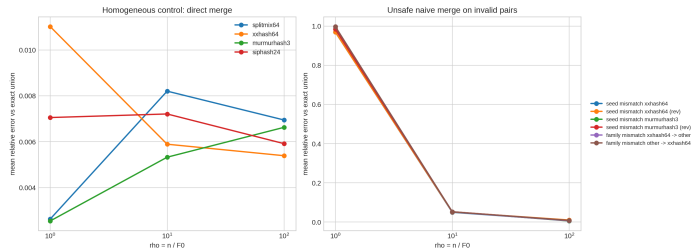
Risultato 3: il merge omogeneo è il controllo del caso valid

- Configurazione: HyperLogLog++, $\rho \in \{1, 10, 100\}$, $k \in \{10, 14, 18\}$, quattro funzioni di hash.
- Condizione di omogeneità: stessa funzione di hash, stesso seed e stessi parametri sui due lati.
- Totale osservato: 36 configurazioni e 900 coppie di merge.

$$\Delta_{abs} = 0, \quad \Delta_{rel} = 0$$

ρ	d	Coppie totali	Coppie con delta $\neq 0$	Max Δ_{rel}
1	10^7	300	0	0
10	10^6	300	0	0
100	10^5	300	0	0

Risultato 4: hash mismatch = caso invalid

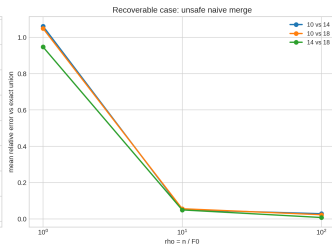
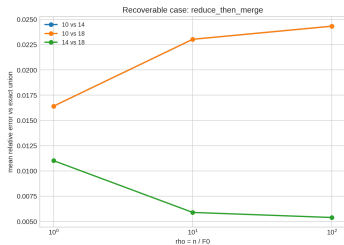


- Scenari osservati: seed mismatch su xxhash64/murmurhash3 e family mismatch tra le quattro hash supportate.
- Controllo omogeneo locale: errore medio = 0.006233.
- Casi invalid: errore medio tra 0.342374 e 0.352802.
- Errore massimo osservato: 1.009634.

Conclusione

Se cambia la semantica dell'hash, il merge forzato perde significato. In questo contesto il caso va rifiutato.

Risultato 5: precision mismatch = caso recoverable



- Hash fissata a xxhash64, stesso seed, coppie (10, 14), (10, 18), (14, 18) e versioni invertite.
- reject: nessuna stima di merge, riferimento seriale = 0.016651.

Strategia	$\bar{\epsilon}_{merge}$
unsafe_naive_merge	0.363882
reduce_then_merge	0.016651

- reduce_then_merge coincide con il riferimento seriale e con il baseline omogeneo a precisione minima.
- Il merge forzato raggiunge massimo 1.061378 e degrado medio 32.37.

Sintesi dei risultati e contributi

Risultati empirici

- HLL++ è il metodo più stabile nel dominio osservato.
- Il merge omogeneo coincide col riferimento seriale.
- L'eterogeneità dell'hash rompe la correttezza del merge.
- Il mismatch di precisione è recuperabile solo tramite normalizzazione esplicita.

Contributi del lavoro

- Implementazione uniforme di più algoritmi count-distinct.
- Interfaccia comune per hash, dataset, metriche e serializzazione.
- Framework riutilizzabile per streaming e merge distribuito.
- Lessico operativo `valid/recoverable/invalid`.

Take-away

Negli sketch count-distinct distribuiti, hash e parametri fanno parte della semantica dello sketch: senza compatibilità semantica il merge non è un'operazione neutra.

Limiti e sviluppi futuri

Limiti del lavoro

- Dataset soprattutto sintetico e controllato
- Studio del merge concentrato soprattutto su casi pairwise
- Analisi eterogenea limitata a hash mismatch e precision mismatch
- Parte di merge sviluppata soprattutto su HLL++

Direzioni future

- Dati reali e integrazione con sorgenti come Apache Kafka
- Topologie distribuite più ampie e scelta del punto ottimale di riduzione alla precisione minima
- Altri assi di eterogeneità semanticamente chiari
- Algoritmi più recenti come UltraLogLog, SetSketch, HyperLogLogLog
- Estensione del framework a Bloom filter, Count-Min Sketch e ad altri costi di merge

Grazie per l'attenzione.